

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

Benemérita Universidad Autónoma de Puebla

PROYECTO FINAL

Materia: Servicios Web

Sistema de Gestión y Automatización de Calificaciones

(AGM – Academic Grade Management)

Arquitectura de Microservicios · gRPC + REST · Angular 20 (Punto Extra) · Despliegue en la nube

Docentes propietarios	Luis Yael Méndez Sánchez / Gustavo Emilio Mendoza Olguín
Asignatura	Servicios Web
Tipo de entregable	Proyecto Final – Evaluación Integradora (30% de calificación final)
Tecnología Backend	Libre elección por microservicio (Django REST / FastAPI / Express.js / NestJS / Spring Boot)
Comunicación entre MS	gRPC (obligatorio entre microservicios) + REST (hacia el cliente)
Arquitectura	Microservicios independientes con BD propia cada uno
Frontend Angular 20	PUNTO EXTRA – Actividad opcional para +1 punto sobre calificación final
Integrantes por equipo	Máximo 6 personas

ÍNDICE DE CONTENIDO

1. Título del proyecto
2. Objetivo general
3. Objetivos específicos
4. Instrucciones generales
5. Descripción detallada del proyecto
 - 5.1 Contexto y alcance del sistema
 - 5.2 Roles del sistema y sus funcionalidades
 - 5.3 Módulos funcionales requeridos
 - 5.4 Arquitectura de microservicios – Backend (NÚCLEO DEL PROYECTO)
 - 5.4.1 Los 7 microservicios principales
 - 5.4.2 Base de datos independiente por microservicio
 - 5.4.3 Comunicación con gRPC
 - 5.4.4 Propuesta de tecnologías backend
 - 5.4.5 Estándares de diseño de la API
 - 5.4.6 Despliegue del backend en la nube
 - 5.5 Frontend Angular 20 – Actividad Opcional (Punto Extra)
6. Entregables
7. Criterios de evaluación

1. TÍTULO DEL PROYECTO

Sistema de Gestión y Automatización de Calificaciones (AGM)

Desarrollo de Backend con Arquitectura de Microservicios y gRPC

2. OBJETIVO GENERAL

Desarrollar la aplicación web Sistema de Gestión y Automatización de Calificaciones (AGM) para la Facultad de Ciencias de la Computación de la BUAP, construyendo su backend bajo una arquitectura de microservicios independientes que se comuniquen entre sí mediante gRPC, con despliegue en un entorno de nube accesible públicamente.

El proyecto se centra en el diseño e implementación del backend como núcleo del sistema, abarcando los servicios esenciales para la gestión académica: control de periodos, materias, alumnos, calificaciones, asistencias QR, notificaciones y reportes. La calidad de la arquitectura de microservicios, la correcta separación de responsabilidades entre servicios, el uso de gRPC para la comunicación inter-servicio y la completitud funcional de cada microservicio son los ejes de evaluación principales de este proyecto.

3. OBJETIVOS ESPECÍFICOS

1. **Implementar siete microservicios independientes** (Auth, Periodos/Materias, Docentes/Alumnos, Calificaciones, Asistencias QR, Notificaciones y Reportes), cada uno con su propia base de datos, lógica de negocio encapsulada y proceso de ejecución autónomo.
2. **Establecer la comunicación inter-servicio mediante gRPC** definiendo los contratos en archivos .proto para todos los flujos que requieren intercambio de datos entre microservicios, garantizando el bajo acoplamiento e interoperabilidad entre servicios.
3. **Proteger todos los recursos del sistema** mediante autenticación basada en JSON Web Tokens (JWT) y control de acceso por roles (RBAC), asegurando que únicamente los usuarios con el perfil correspondiente (Administrador, Docente o Alumno) ejecuten las operaciones reservadas a su rol.
4. **Automatizar la importación de datos académicos** procesando archivos PDF de programación académica oficial y archivos Excel/CSV de listas de alumnos dentro del microservicio correspondiente, normalizando y persistiendo la información sin intervención manual del administrador.
5. **Modelar la base de datos de cada microservicio** de forma independiente, eligiendo el motor más adecuado a la naturaleza de sus datos (relacional o no relacional) y documentando el esquema completo con diccionario de datos en el Manual Técnico.

6. **Desplegar todos los microservicios en un entorno de nube pública** mediante contenedores Docker, garantizando que cada servicio sea accesible con URL pública y HTTPS, reproducible localmente con un único comando docker-compose.
7. **Documentar la arquitectura completa del backend** mediante un Manual Técnico que incluya el diagrama de microservicios, los contratos gRPC (.proto), los endpoints REST externos, el modelo de datos de cada servicio y la guía de instalación paso a paso.

4. INSTRUCCIONES GENERALES

4.1 Organización de equipos

- Los equipos de trabajo deberán estar conformados por un mínimo de 4 y un máximo de 6 integrantes.
- Cada integrante deberá tener un rol claramente definido: líder de proyecto, desarrollador de microservicios (uno o más), DBA/arquitecto de datos, DevOps/despliegue, QA/testing. Los roles deben quedar documentados en el README del repositorio.
- Todos los integrantes deben conocer el sistema completo; durante la presentación se realizarán preguntas a cualquier miembro del equipo.

4.2 Tecnologías y restricciones

- El backend DEBE implementarse bajo arquitectura de microservicios. No se aceptará un monolito único.
- La comunicación entre microservicios DEBE realizarse mediante gRPC usando archivos de definición .proto. La comunicación hacia el cliente externo (navegador o herramienta de pruebas) puede ser REST/HTTP.
- Cada microservicio DEBE tener su propia base de datos. Aunque todos usen el mismo motor (ej. PostgreSQL), cada servicio debe tener su esquema o instancia separada, sin acceder directamente a la BD de otro servicio.
- La tecnología de backend por microservicio es libre (ver propuestas en sección 5.4.4). El equipo debe justificar su elección en el Manual Técnico.
- El frontend en Angular 20 es OPCIONAL y otorga un punto extra sobre la calificación final del semestre. Se describe en la sección 5.5.

4.3 Control de versiones

- Todo el desarrollo debe estar versionado en GitHub con ramas main (producción) y develop (integración).
- Se evaluará el historial de commits: se espera contribución continua durante el semestre, no solo al final.
- La entrega se realiza mediante la URL del repositorio público y las URLs de los servicios desplegados en producción.

4.4 Originalidad

- Queda estrictamente prohibido usar código fuente de proyectos anteriores. Los manuales proporcionados son únicamente documentos de requerimientos.
 - El uso de herramientas de IA está permitido siempre que el equipo comprenda y pueda explicar todo el código generado.
-

5. DESCRIPCIÓN DETALLADA DEL PROYECTO

5.1 Contexto y Alcance del Sistema

El Sistema de Gestión y Automatización de Calificaciones (AGM) es una plataforma académica digital diseñada para apoyar a los docentes de la Facultad de Ciencias de la Computación de la BUAP en el control integral de sus cursos. El sistema centraliza los procesos de registro de alumnos, asignación de ponderaciones de calificación, control de asistencias mediante código QR, seguimiento de entregas, generación de reportes y visualización de estadísticas históricas.

Actualmente existe una versión básica del sistema desarrollada en ciclos anteriores. Para este proyecto, los alumnos deberán construir desde cero el backend completo bajo arquitectura de microservicios, tomando los manuales proporcionados como documento de requerimientos funcionales de referencia. El enfoque principal del proyecto es el diseño, implementación, comunicación y despliegue de los microservicios; el frontend es un componente secundario opcional.

FOCO DEL PROYECTO: Nota importante: este proyecto evalúa competencias de Servicios Web. El núcleo técnico es la construcción del backend con microservicios independientes que se comunican mediante gRPC. Un proyecto con backend incompleto pero buen frontend no obtendrá buena calificación. Un proyecto con backend sólido, bien documentado y desplegado, aunque sin frontend, obtendrá una calificación excelente.

5.2 Roles del Sistema y sus Funcionalidades

El sistema contempla tres roles de usuario. Comprender estas funcionalidades es esencial para diseñar correctamente los contratos de cada microservicio, ya que cada operación del sistema corresponde a al menos una operación en alguno de los servicios del backend.

5.2.1 Rol: Administrador

El Administrador es el usuario con mayor nivel de privilegios. Su función principal es configurar el entorno académico para cada semestre. Sus operaciones cubren:

- Gestión completa de periodos académicos: crear, editar, activar/desactivar y eliminar periodos. El sistema debe garantizar que solo exista un periodo activo en todo momento, ya que es la referencia para todos los procesos académicos vigentes.
- Importación masiva de materias desde el archivo PDF oficial de programación académica de Secretaría Académica. El microservicio de importación debe procesar el PDF, extraer NRC, nombre de materia, sección, docente asignado y horario, y registrar todo automáticamente.
- Importación masiva del directorio de docentes desde el PDF institucional. El sistema extrae nombre completo, correo institucional y cubículo de cada docente.
- Gestión del catálogo de docentes: visualización, búsqueda, paginación y restablecimiento de contraseñas.
- Dashboard con información del periodo activo, fecha y hora del sistema, y acceso a las funciones de gestión.

5.2.2 Rol: Docente

El Docente gestiona el proceso académico de sus materias. Sus funcionalidades son las más amplias y abarcan todo el ciclo de evaluación:

- Dashboard personalizado con estadísticas en tiempo real: total de materias asignadas, total de alumnos inscritos, porcentaje de asistencia del día y tabla resumen de materias.
- Gestión de materias: visualización de las materias asignadas en el periodo activo con su NRC, sección y estado. Importación de alumnos desde Excel/CSV con vista previa antes de confirmar. Al importar un alumno por primera vez, el sistema envía automáticamente al alumno su clave única de acceso por correo electrónico.
- Configuración de ponderaciones (criterios de evaluación) por materia: el docente define categorías como Exámenes 40%, Tareas 30%, Proyectos 20%, Asistencia 10%, con validación de que la suma sea exactamente 100%. Se puede configurar manualmente o importar desde Excel.
- Gestión de actividades y calificaciones: para cada categoría de ponderación, el docente crea actividades específicas (ej. 'Examen Parcial 1', 'Proyecto Final') y asigna calificaciones individuales a cada alumno de forma manual o por importación Excel. El concentrado debe mostrar el promedio ponderado real y el promedio redondeado (fracción ≥ 0.5 redondea al entero superior; < 0.5 al entero inferior).
- Módulo de pase de lista / asistencia QR: el docente inicia una sesión de 10 minutos para una materia. Durante ese tiempo la cámara del dispositivo escanea los códigos QR dinámicos de los alumnos. El sistema clasifica la asistencia como 'Presente' (primeros 5 minutos) o 'Retardo' (entre 5 y 10 minutos). La sesión se cierra automáticamente al llegar a cero.
- Cierre de materia: al cerrar una materia el sistema notifica por correo a todos los alumnos inscritos. Una materia cerrada puede recibir modificaciones hasta que se imprima la lista final. Al imprimir la lista final ya no se permiten más cambios.
- Exportación de calificaciones finales en formato Excel (XLS) y PDF para captura en actas institucionales.
- Historial académico: materias impartidas por periodo, con indicadores visuales para periodos inactivos y estadísticas comparativas si la misma materia se ha impartido en más de un periodo.
- Proceso de pase de lista: el docente puede confirmar la lista o solicitarla de nuevo ante alguna irregularidad.

5.2.3 Rol: Alumno

El Alumno accede con la clave única enviada por correo al momento de su registro. Sus funcionalidades son de consulta y asistencia:

- Dashboard con mensaje de bienvenida personalizado, nombre, matrícula, tipo de formación y periodo activo. Tabla de materias actuales con NRC, nombre, docente y sección.
- Consulta de materias: listado de materias inscritas, detalle de actividades y calificaciones parciales, promedio ponderado actual.
- Baja de materia: el alumno puede darse de baja de una materia únicamente una vez. La operación es irreversible y genera automáticamente una notificación por correo al docente. Una vez dado de baja, el alumno pierde todo acceso a esa materia.
- Módulo de asistencia QR: genera dinámicamente su código QR personal con matrícula, identificador de sesión y marca de tiempo cifrada. El QR se regenera cada pocos segundos para evitar capturas y uso fraudulento.
- Estadísticas de asistencias y entregas por materia inscrita.

5.3 Módulos Funcionales Requeridos

Los siguientes módulos funcionales deben estar cubiertos por el conjunto de microservicios del backend. Cada módulo se describe aquí en términos de negocio; su implementación técnica se detalla en la sección 5.4.

Módulo 1 – Autenticación y Seguridad

Gestiona el acceso al sistema para los tres tipos de usuario. Incluye: inicio de sesión con correo y contraseña, generación y validación de tokens JWT con expiración, recuperación de contraseña vía correo con enlace de un solo uso, y cierre de sesión con invalidación del token. El middleware de autorización verifica en cada solicitud el token y el rol del usuario para determinar el acceso al recurso.

Módulo 2 – Gestión de Periodos Académicos

Permite al Administrador crear, editar, activar/desactivar y eliminar periodos académicos. Un periodo contiene nombre, fecha de inicio, fecha de fin y plan de estudios. El sistema valida que solo exista un periodo activo a la vez. Es el módulo más crítico porque todos los demás procesos académicos dependen del periodo activo como referencia temporal.

Módulo 3 – Importación y Gestión de Materias y Docentes

El microservicio de importación recibe un archivo PDF oficial de programación académica y, mediante técnicas de parsing y procesamiento de texto, extrae automáticamente NRC, nombre de materia, sección, clave, docente asignado y horario. También procesa el PDF del directorio institucional para extraer la información de cada docente. El módulo también gestiona las operaciones CRUD manuales sobre materias y docentes.

Módulo 4 – Gestión de Alumnos y Ponderaciones

Gestiona la inscripción de alumnos por importación Excel/CSV, el envío automático de correos de bienvenida con clave única, la visualización del concentrado de alumnos por materia y la configuración de ponderaciones (criterios de evaluación) por materia con validación de que la suma sea 100%.

Módulo 5 – Calificaciones y Actividades

El docente crea actividades bajo cada categoría de ponderación, asigna calificaciones individuales o masivas (por importación Excel), y el sistema calcula automáticamente el promedio ponderado de cada alumno. El concentrado final muestra el promedio real y el redondeado según las reglas institucionales (≥ 0.5 redondea al entero superior; < 0.5 al inferior).

Módulo 6 – Asistencias QR

El alumno genera un código QR dinámico con su información cifrada y una marca de tiempo. El docente inicia una sesión de 10 minutos en la que escanea los QR desde la cámara del dispositivo. El backend valida el QR, verifica que no haya sido usado previamente (anti-duplicados), registra la asistencia y determina el estado (Presente o Retardo) según la marca de tiempo comparada con el inicio de la sesión. Incluye estadísticas de sesión en tiempo real.

Módulo 7 – Notificaciones y Correos

Microservicio dedicado al envío de correos electrónicos transaccionales: clave única al alumno al registrarse, notificación al docente cuando un alumno solicita baja, notificación a alumnos cuando se cierra una materia y se publican calificaciones, y enlace de recuperación de contraseña.

Módulo 8 – Reportes y Exportaciones

Generación y descarga de reportes en formato Excel (XLS/XLSX) y PDF: lista de calificaciones finales por materia, concentrado de asistencias, y estadísticas de rendimiento. El microservicio recibe los datos y genera los archivos usando librerías especializadas.

Módulo 9 – Estadísticas e Historial

Estadísticas de asistencias y entregas a nivel individual (alumno) y grupal (materia), tanto del periodo activo como históricas. El docente puede comparar el rendimiento de una misma materia impartida en diferentes periodos. El alumno visualiza sus propias estadísticas.

5.4 Arquitectura de Microservicios – Backend (NÚCLEO DEL PROYECTO)

NÚCLEO DEL PROYECTO: Sección principal del proyecto: el backend debe estar dividido en microservicios genuinamente independientes — procesos separados, puertos distintos, bases de datos propias, Dockerfiles individuales. No es válido tener un único servidor con rutas separadas; cada microservicio debe poder ejecutarse, escalarse y desplegarse de forma completamente autónoma.

5.4.1 Los 7 Microservicios Principales

A continuación se definen los siete microservicios que el equipo DEBE implementar. Para cada uno se especifica su responsabilidad, los recursos REST que expone hacia el exterior (cliente/API Gateway) y los métodos gRPC que expone hacia otros microservicios internos.

Microservicio	Responsabilidad principal	Recursos REST externos (hacia el cliente)	Métodos gRPC internos (hacia otros microservicios)
MS-1 Auth & Users	Autenticación JWT, gestión de credenciales, RBAC, recuperación de contraseña. Centraliza la identidad de todos los usuarios del sistema.	POST /auth/login POST /auth/refresh-token POST /auth/forgot-password POST /auth/reset-password GET /auth/me	ValidateToken(token) → UserClaims GetUserById(userId) → UserProfile CheckRole(userId, role) → bool
MS-2 Periodos & Materias	CRUD de periodos académicos, importación PDF de programación, gestión del catálogo de materias por periodo, validación de periodo único activo.	GET/POST/PUT/DELETE /periodos POST /periodos/importar GET /materias?periodo=:id GET /materias/:id	GetMateriaById(materialId) → MaterialInfo GetMateriasByDocente(docentId) → [Materia] GetPeriodoActivo() → PeriodoInfo
MS-3 Docentes & Alumnos	Importación PDF de directorio docente, CRUD de docentes,	POST /docentes/importar GET /docentes POST /alumnos/importar/:materialId GET /alumnos/materia/:materialId DELETE /alumnos/:id/baja	GetAlumnosByMateria(materialId) → [AlumnoInfo] GetAlumnoById(alumnoId) → AlumnoInfo IsAlumnoEnMateria(alumnoId, materialId) → bool

Microservicio	Responsabilidad principal	Recursos REST externos (hacia el cliente)	Métodos gRPC internos (hacia otros microservicios)
	importación Excel de alumnos por materia, gestión del concentrado de alumnos, baja de materia.		
MS-4 Calificaciones & Ponderaciones	Gestión de ponderaciones (criterios 100%), registro de actividades por categoría, asignación de calificaciones individuales o masivas, cálculo de promedios ponderados y redondeados.	GET/POST/PUT /ponderaciones/:materiald POST /actividades POST /calificaciones POST /calificaciones/importar GET /concentrado/:materiald	GetConcentrado(materiald) → [AlumnoCalif] GetPromedioAlumno(alumnoId, materiald) → float GetEstadisticasMateria(materiald) → Stats
MS-5 Asistencias QR	Gestión de sesiones de asistencia (10 min), validación de tokens QR dinámicos con anti-replay, clasificación Presente/Retardo, estadísticas diarias y por sesión.	POST /sesiones/iniciar POST /asistencias/registrar DELETE /sesiones/:id/cerrar GET /asistencias/:materiald/hoy GET /asistencias/:materiald/historial	GetAsistenciaAlumno(alumnoId, materiald) → [Asistencia] GetEstadisticasAsistencia(materiald) → Stats
MS-6 Notificaciones	Envío de correos transaccionales: clave única al registrar alumno, alerta de baja al docente, aviso de calificación final a	POST /notificaciones/bienvenida POST /notificaciones/baja POST /notificaciones/cierre-materia POST /notificaciones/reset-password	SendBienvenida(alumnoId, materiald) → bool SendBajaNotif(alumnoId, docenteId) → bool SendCierreMateria(materiald) → bool

Microservicio	Responsabilidad principal	Recursos REST externos (hacia el cliente)	Métodos gRPC internos (hacia otros microservicios)
	alumnos, enlace de recuperación de contraseña.		
MS-7 Reportes & Estadísticas	Generación de archivos Excel/PDF de calificaciones finales y asistencias, estadísticas históricas de rendimiento grupal e individual, comparativa entre periodos.	GET /reportes/calificaciones/:materiald?formato=pdf xls GET /reportes/asistencias/:materiald?formato=pdf xls GET /estadisticas/docente/:id GET /estadisticas/alumno/:id	GenerateReport(params) → FileBytes GetHistorialDocente(docentelid) → [StatsPeriodo]

Nota: Aclaración sobre la tabla: los "Recursos REST externos" son las rutas HTTP que el microservicio expone al cliente (navegador, Postman, API Gateway). Los "Métodos gRPC internos" son los procedimientos que ese microservicio ofrece a otros microservicios del sistema para comunicación directa inter-servicio. Ningún microservicio debe acceder directamente a la base de datos de otro; si necesita datos de otro servicio, lo solicita mediante gRPC.

5.4.2 Base de Datos Independiente por Microservicio

Uno de los principios fundamentales de la arquitectura de microservicios es el aislamiento de datos. Cada microservicio es el único propietario de su base de datos y ningún otro servicio puede acceder a ella directamente. Cuando un microservicio necesita datos de otro, los solicita a través de su interfaz gRPC definida.

Principio clave: Ejemplo de aislamiento de datos: si el MS-4 (Calificaciones) necesita el nombre de un alumno para generar el concentrado, no accede a la base de datos del MS-3 (Alumnos). En cambio, realiza una llamada gRPC: GetAlumnoById(alumnoid) al MS-3, quien responde con los datos solicitados. Este principio garantiza el desacoplamiento real entre servicios.

A continuación se describe la base de datos recomendada para cada microservicio, justificada por la naturaleza de sus datos:

Microservicio	BD recomendada	Motor sugerido	Justificación
MS-1 Auth & Users	Relacional	PostgreSQL	Las relaciones usuario-rol son bien estructuradas; necesita

Microservicio	BD recomendada	Motor sugerido	Justificación
			transacciones ACID para el manejo seguro de credenciales.
MS-2 Periodos & Materias	Relacional	PostgreSQL / MySQL	Relaciones complejas entre periodos, planes de estudio y materias con integridad referencial.
MS-3 Docentes & Alumnos	Relacional	PostgreSQL / MySQL	Relaciones claras entre docentes, alumnos y materias. Búsquedas frecuentes por NRC o matrícula.
MS-4 Calificaciones	Relacional documental o	PostgreSQL / MongoDB	Las ponderaciones y actividades varían por materia; MongoDB ofrece flexibilidad para estructuras variables por docente.
MS-5 Asistencias QR	Relacional + caché	PostgreSQL + Redis	Las sesiones activas de asistencia son datos volátiles de corta duración, ideales para Redis. Los registros históricos van en PostgreSQL.
MS-6 Notificaciones	Documental relacional o	MongoDB / PostgreSQL	Historial de correos enviados con estructura flexible (distintos templates y parámetros). MongoDB se adapta bien.
MS-7 Reportes & Stats	Relacional analítica	PostgreSQL	Consultas de agregación complejas sobre datos históricos; PostgreSQL con vistas materializadas es óptimo.

Nota: aunque dos microservicios usen el mismo motor (ej. PostgreSQL), deben tener bases de datos separadas con nombres distintos (ej. agm_auth_db, agm_periodos_db, agm_alumnos_db). En el docker-compose.yml se pueden declarar múltiples instancias de contenedores de base de datos, o bien usar una instancia con múltiples bases de datos separadas. Lo que nunca debe ocurrir es que un servicio ejecute queries contra el esquema de otro.

5.4.3 Comunicación entre Microservicios con gRPC

gRPC (Google Remote Procedure Call) es el protocolo obligatorio para la comunicación interna entre microservicios. A diferencia de REST, gRPC utiliza Protocol Buffers (.proto) para definir los contratos de comunicación de forma estricta, lo que garantiza interoperabilidad y alta eficiencia en la serialización de datos.

El equipo debe crear un archivo .proto por cada microservicio que expone métodos gRPC hacia otros. Estos archivos deben estar versionados en el repositorio (carpeta /proto en la raíz) y ser compartidos entre los servicios que los consumen. A continuación se describen las consideraciones técnicas clave:

- Definición de servicios: cada archivo .proto define un service con sus rpc (procedimientos remotos), los mensajes de entrada (request) y salida (response).

- Generación de código: a partir del .proto se genera automáticamente el código cliente y servidor en el lenguaje elegido (Python con grpcio-tools, JavaScript/TypeScript con @grpc/grpc-js y @grpc/proto-loader, Java con el plugin de Maven/Gradle).
- Comunicación unidireccional y streaming: para la mayoría de las operaciones del sistema es suficiente el RPC unario (una solicitud, una respuesta). Para el módulo de asistencias en tiempo real se puede explorar el streaming del servidor (server-side streaming).
- Manejo de errores: gRPC usa códigos de estado propios (OK, NOT_FOUND, PERMISSION_DENIED, INTERNAL) que deben manejarse correctamente en los servicios consumidores.
- Puertos: cada microservicio debe tener un puerto dedicado para su servidor gRPC (diferente al puerto REST). Por convención se recomienda usar el rango 50051-50057 para los servidores gRPC internos.

5.4.4 Propuesta de Tecnologías para el Backend

Cada microservicio puede implementarse con la tecnología que mejor domine el equipo. A continuación se presentan las opciones recomendadas con sus ventajas para este proyecto:

Tecnología	Lenguaje	Ventajas para este proyecto	Microservicios recomendados
Django REST Framework	Python	ORM robusto, admin integrado, excelente para APIs con lógica compleja, gran soporte para procesamiento de archivos PDF/Excel	MS-1, MS-2, MS-3, MS-4
FastAPI	Python	Alto rendimiento asíncrono, documentación OpenAPI automática, integración nativa con grpcio, ideal para servicios de alta concurrencia	MS-5, MS-6, MS-7
Express.js / NestJS	JavaScript / TypeScript	Ecosistema npm amplio, @grpc/grpc-js bien documentado, Nodemailer para notificaciones, NestJS tiene módulos gRPC nativos	MS-1, MS-5, MS-6
Spring Boot	Java	Soporte nativo para gRPC con grpc-spring-boot-starter, escalabilidad empresarial, ideal para servicios con lógica compleja y datos críticos	MS-2, MS-4
Laravel (API)	PHP	Rápido de implementar para CRUDs, Eloquent ORM, buena opción si el equipo tiene experiencia previa en PHP	MS-3, MS-4

5.4.5 Estándares de Diseño de la API REST Externa

Los recursos REST expuestos hacia el exterior (cliente o API Gateway) deben seguir estos estándares:

- Verbos HTTP correctos: GET para consultar, POST para crear, PUT/PATCH para actualizar, DELETE para eliminar.
- Códigos de estado apropiados: 200 (éxito), 201 (recurso creado), 400 (datos inválidos), 401 (no autenticado), 403 (sin permisos), 404 (no encontrado), 500 (error interno).
- Todas las respuestas en formato JSON con estructura consistente: { "success": true, "data": {...}, "message": "" }.
- Paginación en todos los endpoints que retornen listas: parámetros ?page=1&limit=10.
- Validación de todos los datos de entrada en el backend, independientemente de si hay frontend o no.
- Documentación con Swagger/OpenAPI o colección Postman exportada, incluida en el Manual Técnico y en el repositorio.
- API Gateway (opcional pero recomendado): un componente central que actúa como punto de entrada único para el cliente, enrutando las peticiones al microservicio correspondiente. Puede implementarse con Nginx como proxy reverso o con un microservicio propio en Express.js.

5.4.6 Despliegue del Backend en la Nube

Todos los microservicios deben estar desplegados y accesibles públicamente al momento de la presentación. Se recomienda fuertemente dockerizar cada microservicio con su propio Dockerfile. El repositorio debe incluir un docker-compose.yml en la raíz que permita levantar todo el sistema localmente con un solo comando.

Plataformas de despliegue recomendadas por su plan gratuito adecuado para este tipo de proyectos:

Plataforma	Tipo	Ventajas	Plan gratuito
Railway	PaaS (Backend + BD)	Despliegue directo desde GitHub, soporte Docker, PostgreSQL administrado incluido, ideal para múltiples microservicios	Sí (con límites)
Render	PaaS (Backend + BD)	Despliegue automático desde GitHub, soporta Docker, servicios web y bases de datos administradas, fácil de configurar	Sí (con sleep)
Fly.io	Containers	Despliegue de contenedores Docker, buena latencia global, control total del entorno de red	Sí (limitado)
AWS / GCP / Azure	Cloud completo	Máxima flexibilidad, créditos educativos disponibles, ideal	Créditos educativos

Plataforma	Tipo	Ventajas	Plan gratuito
		para equipos con experiencia en cloud	

- Variables de entorno: todas las configuraciones sensibles (cadenas de conexión, claves JWT, credenciales SMTP, claves gRPC) deben manejarse como variables de entorno. El repositorio debe incluir un .env.example por cada microservicio.
- CORS: cada microservicio debe configurar CORS para aceptar peticiones únicamente desde los orígenes autorizados (otros microservicios y el frontend, si aplica).
- HTTPS: todas las URLs de producción deben usar HTTPS. Las plataformas recomendadas lo proveen automáticamente.

5.5 Frontend Angular 20 – Actividad Opcional (Punto Extra)

Punto extra: Actividad opcional: el desarrollo del frontend en Angular 20 no forma parte del proyecto obligatorio. Es una actividad complementaria para los equipos que deseen obtener +1 punto adicional sobre su calificación final del semestre. Este punto extra se otorga únicamente si el frontend está completo, funcional, conectado a los microservicios del backend desplegado en producción, y tiene un diseño de interfaz profesional. Un frontend parcial o sin conexión real al backend no otorga el punto.

El frontend debe desarrollarse como una Single Page Application (SPA) en Angular (última versión estable disponible). Consumirá los microservicios del backend mediante HTTP/REST, con autenticación JWT en todos los headers de las peticiones. A continuación se describen los requisitos mínimos para que el frontend sea considerado completo y acreedor al punto extra:

5.5.1 Requisitos Técnicos Angular

- Lazy Loading: cada módulo (admin, docente, alumno) debe cargarse de forma diferida para optimizar el tiempo de carga inicial.
- Route Guards: implementar CanActivate para proteger rutas según el rol del usuario autenticado. Un alumno no debe poder acceder a rutas de administrador.
- HTTP Interceptors: interceptor que agregue automáticamente el token JWT al header de todas las peticiones salientes, y otro que maneje los errores 401 redirigiendo al login.
- Reactive Forms con validaciones síncronas y asíncronas para todos los formularios del sistema.
- Generación de QR en el módulo del alumno usando angularx-qrcode u equivalente. Escaneo de QR en el módulo del docente usando jsQR o zxing-js con acceso a la cámara (MediaDevices API).
- Variables de entorno: las URLs de los microservicios deben configurarse desde src/environments/environment.ts, no hardcodeadas.

5.5.2 Requisitos de Diseño

- Identidad visual consistente: paleta de colores definida, tipografía uniforme, iconografía coherente. Usar una librería de componentes UI: Angular Material, PrimeNG, Taiga UI o ng-zorro-antd.
- Diseño completamente responsivo (mobile-first). El módulo de asistencias QR debe funcionar correctamente desde dispositivos móviles.
- Dashboard del docente con gráficas (Chart.js, ApexCharts o ng2-charts) para estadísticas de asistencia y rendimiento.
- Todos los formularios con retroalimentación visual: mensajes de error inline, spinners de carga, confirmaciones de éxito.
- Tablas con paginación, búsqueda en tiempo real y ordenamiento por columnas.
- NO se acepta Bootstrap básico sin personalización, tablas HTML sin estilo, ni formularios sin validación visual.

5.5.3 Criterio de Otorgamiento del Punto Extra

Para otorgar el punto extra, el frontend deberá cumplir TODOS los siguientes criterios en la presentación:

Criterio	Otorga el punto extra
Frontend completo con todos los módulos funcionales y conectado al backend en producción	Sí
Frontend parcial (solo algunos módulos o roles implementados)	No
Frontend completo pero sin conexión real al backend (datos mock/hardcodeados)	No
Frontend completo pero con diseño genérico sin personalización	No
Frontend completo, funcional, conectado y con diseño profesional	Sí (+1 punto sobre calificación final del semestre)

6. ENTREGABLES

Todos los entregables son obligatorios para el proyecto base (backend). El frontend Angular es adicional. La ausencia de cualquier entregable obligatorio impacta directamente en la calificación.

6.1 Manual de Usuario

Documento profesional en formato PDF o Word que describe el funcionamiento de la aplicación desde el punto de vista del usuario final. Debe incluir:

- Portada con nombre del proyecto, nombre del equipo e integrantes, materia y fecha.
- Introducción y propósito del sistema.
- Descripción de cómo acceder e interactuar con el sistema (ya sea desde el frontend si se desarrolló, o desde una interfaz de prueba como Swagger UI o Postman si solo se entrega el backend).
- Sección por cada rol (Administrador, Docente, Alumno): descripción paso a paso de cada funcionalidad con capturas de pantalla o evidencias del sistema en producción.
- Diseño profesional: portada estilizada, índice con hipervínculos, numeración de páginas, encabezados y pies de página coherentes.

6.2 Manual Técnico

Documento técnico detallado para desarrolladores y administradores. Debe incluir:

- Diagrama de arquitectura de microservicios: mostrando todos los servicios, sus puertos REST y gRPC, sus bases de datos individuales, y el flujo de comunicación entre ellos.
- Stack tecnológico completo con justificación de la elección por microservicio.
- Modelo de datos: diagrama entidad-relación o de colecciones por cada microservicio, con diccionario de datos (nombre de campo, tipo, restricciones).
- Contratos gRPC: descripción de cada archivo .proto con sus servicios y mensajes definidos.
- Documentación de la API REST de cada microservicio: método HTTP, URL, parámetros, body de request, response esperado, códigos de error. Preferentemente como colección Postman o archivo OpenAPI.
- Guía de instalación local paso a paso: clonar repositorio, configurar variables de entorno, levantar bases de datos, ejecutar todos los microservicios con docker-compose up.
- Guía de despliegue en producción: pasos para desplegar en la plataforma cloud elegida.

6.3 Video Demostrativo

Video de entre 10 y 20 minutos mostrando el funcionamiento completo del sistema. Debe:

- Iniciar con presentación breve del equipo y arquitectura del sistema (máximo 1-2 min).
- Mostrar la arquitectura desplegada: URLs de cada microservicio en producción, estructura del repositorio.
- Demostrar el flujo completo del rol Administrador: autenticación, gestión de periodos, importación de materias y docentes.

- Demostrar el flujo completo del rol Docente: configuración de ponderaciones, importación de alumnos, registro de calificaciones, pase de lista QR, cierre de materia y exportación de reporte.
- Demostrar el flujo completo del rol Alumno: acceso, consulta de calificaciones, generación de QR para asistencia.
- Mostrar el sistema de notificaciones por correo en funcionamiento real.
- Si se desarrolló el frontend: mostrar la aplicación en móvil para demostrar responsividad.
- Subir a YouTube (puede ser privado con enlace compartible) e incluir la URL en el README.

6.4 Repositorio de GitHub

Repositorio público con código fuente completo. Requisitos:

- Estructura monorepo recomendada: una carpeta por microservicio (/ms-auth, /ms-periodos, /ms-alumnos, /ms-calificaciones, /ms-asistencias, /ms-notificaciones, /ms-reportes) y una carpeta /proto con todos los archivos de definición gRPC compartidos. Si se desarrolló el frontend: carpeta /frontend.
- Ramas: main (producción estable) y develop (integración activa). Historial de commits continuo durante el semestre.
- README.md principal con: descripción del proyecto, integrantes y roles, stack tecnológico, prerequisites, instrucciones de instalación local (docker-compose), instrucciones de configuración de variables de entorno, URL de producción de cada microservicio, URL del video demostrativo, y estructura del repositorio explicada.
- .env.example en cada microservicio con todas las variables necesarias (sin valores reales).
- docker-compose.yml en la raíz para levantar todo el sistema localmente.
- Carpeta /proto con todos los archivos .proto versionados.
- Documentación de la API: colección Postman exportada o archivo openapi.yaml por microservicio.

7. CRITERIOS DE EVALUACIÓN

El proyecto final representa el 30% de la calificación final del semestre. La evaluación se basa exclusivamente en el backend con arquitectura de microservicios. La calificación máxima del proyecto es 100 puntos, equivalente al 30% de la calificación final del semestre.

Criterio de evaluación	Peso	Qué se evalúa
1. Arquitectura de microservicios real	30%	Que el backend esté dividido en microservicios genuinamente independientes: procesos separados, puertos distintos, bases de datos propias, Dockerfiles individuales. Se penaliza fuertemente si es un monolito con rutas separadas disfrazado de microservicios.
2. Comunicación gRPC entre servicios	20%	Presencia y correcta definición de archivos .proto en el repositorio, generación de código cliente/servidor, llamadas gRPC funcionales entre al menos tres pares de microservicios, manejo adecuado de errores gRPC. Se revisará durante la presentación con preguntas específicas.
3. Funcionalidad completa del backend	25%	Que todos los módulos funcionales estén operativos: importación de PDFs y Excel, gestión de periodos/materias/alumnos, ponderaciones y calificaciones, sesiones de asistencia QR, notificaciones por correo y exportación de reportes. Se verifica cada flujo mediante llamadas directas a la API.
4. Despliegue en producción	10%	Que todos los microservicios estén desplegados con URLs públicas y HTTPS al momento de la presentación. Uso correcto de variables de entorno, docker-compose.yml funcional, configuración de CORS. Si un servicio no está desplegado, ese criterio otorga 0 puntos.
5. Calidad del código y repositorio	8%	Historial de commits continuo y descriptivo, estructura de carpetas organizada, ausencia de credenciales hardcodeadas, .env.example presente, separación clara de capas (rutas, controladores, servicios, modelos). El repositorio debe evidenciar trabajo durante todo el semestre.
6. Documentación técnica	7%	Calidad y completitud del Manual Técnico (diagrama de arquitectura, contratos .proto, endpoints documentados, modelo de datos, guía de instalación) y del README del repositorio. También se evalúa la claridad del Manual de Usuario.

Criterio de evaluación	Peso	Qué se evalúa
TOTAL	100%	100 puntos = 30% de la calificación final del semestre

7.1 Punto Extra por Frontend Angular

Punto extra — Frontend Angular: +1 punto sobre la calificación final del semestre (no sobre el proyecto) para los equipos que entreguen el frontend en Angular 20, completo, funcional, conectado al backend desplegado en producción y con diseño de interfaz profesional. Los criterios de evaluación completos se encuentran en la sección 5.5.3.

Ejemplo de impacto del punto extra: si un alumno tiene 8.5 de calificación final antes de sumar el proyecto, y su equipo entrega el frontend completo y de calidad, la calificación final sería $8.5 + 1 = 9.5$. El punto extra aplica sobre la calificación final del semestre, independientemente del puntaje obtenido en el proyecto base.

7.2 Criterios de Penalización

- 20 puntos: el backend está implementado como monolito único (un solo servidor, una sola base de datos). Esta es la penalización más severa porque contradice el objetivo central del proyecto.
- 15 puntos: la comunicación entre servicios no usa gRPC sino REST directo entre microservicios.
- 10 puntos: la aplicación no está desplegada en producción o los enlaces no funcionan al momento de la presentación.
- 5 puntos por módulo faltante: cualquier módulo funcional principal que no esté implementado o sea completamente inoperativo.
- 10 puntos: el repositorio tiene menos de 20 commits o fue creado menos de una semana antes de la entrega (evidencia de no haber trabajado durante el semestre).
- 10 puntos: plagio o uso de código de proyectos anteriores sin comprensión demostrada.
- 5 puntos: ausencia de cualquiera de los entregables obligatorios.

7.3 Rúbrica de la Presentación

La presentación es presencial ante el grupo y el docente. Se evaluarán:

Aspecto	Puntos máx.	Criterio
Demo funcional en producción	40	Todos los flujos de usuario funcionan correctamente contra los microservicios desplegados en producción.
Dominio técnico de microservicios y gRPC	35	El equipo puede explicar sus decisiones de arquitectura, mostrar los archivos .proto, describir el flujo de una llamada gRPC entre

Aspecto	Puntos máx.	Criterio
		servicios y responder preguntas técnicas del docente.
Calidad del repositorio y documentación	15	README claro, commits frecuentes, estructura organizada, contratos .proto versionados, documentación de API.
Organización y claridad de la presentación	10	Estructura clara, tiempo bien manejado, todos los integrantes participan y demuestran conocimiento del sistema completo.

Reflexión final: La arquitectura de microservicios con gRPC representa el estado del arte en el desarrollo de sistemas distribuidos en la industria. Al construir este proyecto, los alumnos estarán aplicando los mismos patrones que utilizan compañías como Netflix, Uber, Google y Amazon para escalar sus sistemas. El reto no es solo que el código funcione, sino que la arquitectura esté bien pensada: servicios con responsabilidades claras, contratos bien definidos y datos correctamente aislados. Ese es el nivel de excelencia que se espera de los alumnos de la Facultad de Ciencias de la Computación de la BUAP.